# Universal Logic v2.2.1a — Formal Implementation Specification

**Status:** Patch specification (v2.2.1a), implementation-facing.

**Base:** Universal Logic v2.1+
**Delta:** v2.2 (certificates, adaptive control, semantic profiles) + v2.2.1 (freeze semantics & strict contractivity) + v2.2.1a (proof-level semantics, standardized projector API, deterministic replay hashing, context-free Lipschitz maxima).

---

## 1. Scope and design goals

Universal Logic (UL) provides a typed, multi-logic operator runtime with **fail-closed semantics** and a **contractive safety projection (CSP)** envelope.

v2.2.1a makes one central promise explicit:

- **Formal safety claims are emitted only when backed by formally sound bounds.**

It also makes the runtime *debuggable and replayable* by standardizing projectors, logging certificates, and hashing context deterministically.

---

## 2. Mathematical model

### 2.1 Typed tensor spaces

Each logic module $A$ defines:

- a carrier space $T_A$ (tensors, matrices, probability vectors, etc.),
- a declared norm $\|\cdot\|_A$,
- a safety set $S_A \subseteq T_A$,
- a projector $\Pi_{S_A} : T_A \to S_A$ that is **non-expansive**: $$ |\Pi(x)-\Pi(y)|_A \le |x-y|_A. $$

### 2.2 CSP update map

For an operator $F : T_A \to T_A$ and step size $\alpha \in (0, 1]$, define

$$ G(x) = \Pi\big((1-\alpha)x + \alpha F(x)\big). $$

If $F$ has (formal) Lipschitz upper bound $L_F$ in $\|\cdot\|_A$, then

$$ |G(x)-G(y)|_A \le \big((1-\alpha) + \alpha L_F\big)\,|x-y|_A. $$

Define

$$ \texttt{slope_ub} := (1-\alpha)+\alpha L_F,\qquad \texttt{gap_lb} := 1-\texttt{slope_ub}. $$

A **formally certified contraction** requires `slope_ub < 1 - tol` **and** a formally sound bound.

---

## 3. Core data types and registries

### 3.1 Norm registry (required)

All modules MUST compute residuals and Lipschitz bounds in the same declared metric.

```python
from __future__ import annotations
from dataclasses import dataclass
from typing import Callable, Dict
import numpy as np

class NormRegistry:
    """Maps norm_id -> function that returns a nonnegative scalar."""
    norms: Dict[str, Callable[[np.ndarray], float]] = {}

    @classmethod
    def register(cls, norm_id: str, fn: Callable[[np.ndarray], float]) -> None:
        cls.norms[norm_id] = fn

    @classmethod
    def norm(cls, norm_id: str, arr: np.ndarray) -> float:
        if norm_id not in cls.norms:
            raise KeyError(f"Unknown norm_id: {norm_id}")
        val = float(cls.norms[norm_id](arr))
        if val < 0:
            raise ValueError("Norm returned negative value")
        return val

# Minimal defaults
NormRegistry.register("l2", lambda x: np.linalg.norm(x.ravel(), 2))
NormRegistry.register("linf", lambda x: np.linalg.norm(x.ravel(), np.inf))
NormRegistry.register("frobenius", lambda x: np.linalg.norm(x, "fro"))
```

> **Note (quantum trace norm):** implement trace norm as `sum(singular_values)` for matrices; bind it to `norm_id="trace"` where needed.

### 3.2 TypedTensor (reference)

```
from dataclasses import dataclass
from typing import Any, Dict

FTS = Dict[str, int]  # minimal placeholder; v2.1+ defines full signature
algebra

@dataclass(frozen=True)
class TypedTensor:
    data: Any                 # typically np.ndarray
    sigma: FTS
    algebra: str
    norm_id: str
    safety_set_id: str

    def scaled_add(self, other: "TypedTensor", a: float, b: float) ->
"TypedTensor":
        return TypedTensor(
            data=a * self.data + b * other.data,
            sigma=self.sigma,
            algebra=self.algebra,
            norm_id=self.norm_id,
            safety_set_id=self.safety_set_id,
        )

    def subtract(self, other: "TypedTensor") -> "TypedTensor":
        return TypedTensor(
            data=self.data - other.data,
            sigma=self.sigma,
            algebra=self.algebra,
            norm_id=self.norm_id,
            safety_set_id=self.safety_set_id,
        )

    def norm(self) -> float:
        return NormRegistry.norm(self.norm_id, np.asarray(self.data))
```

---

## 4. Certificates and proof semantics

### 4.1 CSPCertificate (required)

```
from dataclasses import dataclass
from typing import List, Literal, Optional
```

```python
@dataclass
class CSPCertificate:
    # Identification
    op_id: str
    step_index: int
    substep_index: int
    timestamp: float

    # Core contraction metrics
    slope_ub: float
    gap_lb: float
    alpha_used: float
    projection_residual: float

    # Proof semantics
    proof_level: Literal["formal", "heuristic", "none"]
    status: Literal["certified", "accepted_heuristic", "freeze", "rejected",
"error"]

    # Runtime mode/profile
    mode: str
    profile: int
    require_certificates: bool

    # Configuration identifiers
    norm_id: str
    safety_set_id: str
    projector_id: str

    # Adaptive control
    fallback_index: int
    fallback_op_id: str
    backtrack_steps: int
    stagnation_count: int

    # Bound metadata
    bound_soundness: Literal["formal", "heuristic"]
    bound_method: Optional[str]
    bound_confidence: Optional[float]

    # Repair and assumptions
    repairs_applied: List[str]
    assumptions: List[str]
    context_hash: str

    @property
    def is_successful(self) -> bool:
```

```
        return self.status in ["certified", "accepted_heuristic", "freeze"]

    @property
    def is_formally_certified(self) -> bool:
        return self.status == "certified" and self.proof_level == "formal"
```

### 4.2 Determining certification

```
from typing import Tuple

def determine_certification(
    slope_ub: float,
    bound_soundness: str,
    contraction_tolerance: float,
) -> Tuple[str, str]:
    if slope_ub < 1.0 - contraction_tolerance:
        if bound_soundness == "formal":
            return "formal", "certified"
        return "heuristic", "accepted_heuristic"
    return "none", "rejected"
```

---

## 5. Standardized projector API

All projectors MUST return `(Y_projected, repairs, residual)` with residual computed in the tensor's declared norm.

```
from abc import ABC, abstractmethod
from typing import Tuple, List
import numpy as np

class Projector(ABC):
    projector_id: str

    @abstractmethod
    def project(self, Y_raw: TypedTensor, ctx: "Context") -> Tuple[TypedTensor,
List[str], float]:
        ...

class BoxClampProjector(Projector):
    projector_id = "box_clamp_0_1"

    def project(self, Y_raw: TypedTensor, ctx: "Context") -> Tuple[TypedTensor,
List[str], float]:
```

```
        arr = np.asarray(Y_raw.data)
        repairs: List[str] = []
        if np.any(arr < 0):
            repairs.append("clamp_min_0")
        if np.any(arr > 1):
            repairs.append("clamp_max_1")
        proj = np.clip(arr, 0.0, 1.0)
        residual = NormRegistry.norm(Y_raw.norm_id, proj - arr)
        return (
            TypedTensor(
                data=proj,
                sigma=Y_raw.sigma,
                algebra=Y_raw.algebra,
                norm_id=Y_raw.norm_id,
                safety_set_id=Y_raw.safety_set_id,
            ),
            repairs,
            residual,
        )
```

---

## 6. Operator interface and call-path separation

### 6.1 Raw vs CSP-wrapped execution

```
from abc import ABC, abstractmethod
from typing import List, Optional, Tuple

class Operator(ABC):
    """Operators transform TypedTensor within the same module/algebra (unless
explicitly embedding)."""

    op_id: str
    bound_soundness: str = "heuristic"  # MUST be overridden to "formal" when
justified.

    @abstractmethod
    def __call__(self, X: TypedTensor, ctx: Optional["Context"] = None) ->
TypedTensor:
        """Raw operator call (no certification, no backtracking)."""

    @abstractmethod
    def lipschitz(self, ctx: "Context") -> float:
        """Return a Lipschitz upper bound in the declared norm for this
context."""
```

```
    def lipschitz_max(self) -> float:
        """Guaranteed maximum across contexts (conservative by default)."""
        return float("inf")

    def fallback_chain(self) -> List["Operator"]:
        return [self]

    def csp_call(self, X: TypedTensor, ctx: "Context", projector: Projector) ->
Tuple[TypedTensor, CSPCertificate]:
        return csp_step_v2_2_1a(X, self, ctx, projector)
```

---

## 7. Adaptive alpha control (per-operator)

```
from dataclasses import dataclass

@dataclass
class AdaptiveAlphaController:
    op_id: str
    alpha0: float = 1.0
    alpha_min: float = 1e-6
    k_stagnant: int = 5
    learning_rate: float = 0.5

    def __post_init__(self):
        self.alpha = self.alpha0
        self.stagnant_count = 0
        self.rejection_count = 0

    def step_accepted(self, alpha_used: float) -> None:
        if abs(alpha_used - self.alpha_min) < 1e-12:
            self.stagnant_count += 1
        else:
            self.stagnant_count = 0
        self.alpha = min(1.0, alpha_used * (1.0 + self.learning_rate))

    def step_rejected(self, alpha_tried: float) -> None:
        self.rejection_count += 1
        self.alpha = max(self.alpha_min, alpha_tried * self.learning_rate)
        if abs(self.alpha - self.alpha_min) < 1e-12:
            self.stagnant_count += 1

    def should_escalate(self) -> bool:
        return self.stagnant_count >= self.k_stagnant
```

```
    def reset_stagnation(self) -> None:
        self.stagnant_count = 0
        self.rejection_count = 0

    def fallback_exhausted(self, _idx: int) -> None:
        """Optional hook; default no-op."""
        return
```

---

## 8. Bound tightening (formal vs heuristic)

### 8.1 Tightening result

```
from dataclasses import dataclass
from typing import List, Literal, Optional

@dataclass
class BoundTighteningResult:
    lipschitz_bound: float
    soundness: Literal["formal", "heuristic"]
    method: str
    assumptions: List[str]
    confidence: Optional[float] = None
```

### 8.2 Policy (mode-gated)

- **Safe mode:** only formally sound methods are allowed.
- **Balanced/Fast:** heuristic estimators permitted but MUST yield `proof_level="heuristic"` if used.

---

## 9. CSP Algorithm 1′ (v2.2.1a)

### 9.1 Context object (minimal)

```
from dataclasses import dataclass
from typing import List, Optional

@dataclass
class Context:
    mode: str                       # "safe" | "balanced" | "fast"
    profile: int
    require_certificates: bool
```

```
    alpha_controller: AdaptiveAlphaController
    contraction_tolerance: float

    allow_tightening: bool
    bound_tightener: Optional[object] = None

    # Replay / debugging
    step_index: int = 0
    substep_index: int = 0
    seed: int = 0
    assumptions: List[str] = None
    X_history: List[TypedTensor] = None
    heuristic_steps: int = 0
```

### 9.2 Freeze operator (terminal fallback)

```
class FreezeOperator(Operator):
    op_id = "freeze"
    bound_soundness = "formal"

    def is_freeze(self) -> bool:
        return True

    def __call__(self, X: TypedTensor, ctx: Optional[Context] = None) ->
TypedTensor:
        return X

    def lipschitz(self, ctx: Context) -> float:
        return 1.0

    def fallback_chain(self) -> List[Operator]:
        return [self]
```

### 9.3 CSP step implementation

```
import time
import math
from typing import Tuple

def csp_step_v2_2_1a(
    X: TypedTensor,
    op: Operator,
    ctx: Context,
    projector: Projector,
```

```
) -> Tuple[TypedTensor, CSPCertificate]:

    chain = op.fallback_chain()
    start_index = getattr(op, "_current_fallback_index", 0)
    alpha0 = ctx.alpha_controller.alpha
    backtrack_count = 0

    for fallback_idx in range(start_index, len(chain)):
        F_i = chain[fallback_idx]

        # Terminal freeze semantics
        if hasattr(F_i, "is_freeze") and F_i.is_freeze():
            cert = CSPCertificate(
                op_id=op.op_id,
                step_index=ctx.step_index,
                substep_index=ctx.substep_index,
                timestamp=time.time(),
                slope_ub=1.0,
                gap_lb=0.0,
                alpha_used=0.0,
                projection_residual=0.0,
                proof_level="none",
                status="freeze",
                mode=ctx.mode,
                profile=ctx.profile,
                require_certificates=ctx.require_certificates,
                norm_id=X.norm_id,
                safety_set_id=X.safety_set_id,
                projector_id=projector.projector_id,
                fallback_index=fallback_idx,
                fallback_op_id=F_i.op_id,
                backtrack_steps=backtrack_count,
                stagnation_count=ctx.alpha_controller.stagnant_count,
                bound_soundness="formal",
                bound_method=None,
                bound_confidence=None,
                repairs_applied=[],
                assumptions=["Terminal freeze operator: returns input
unchanged"],
                context_hash=compute_context_hash(X, ctx, op.op_id,
projector.projector_id),
            )
            return X, cert

        alpha = alpha0
        while alpha >= ctx.alpha_controller.alpha_min - 1e-15:
            Y_raw = X.scaled_add(F_i(X, ctx), 1.0 - alpha, alpha)
            Y, repairs, residual = projector.project(Y_raw, ctx)
```

```
            # Bound computation
            if ctx.allow_tightening and ctx.bound_tightener is not None:
                br = ctx.bound_tightener.tighten(F_i, ctx.X_history or [], ctx)
                L = float(br.lipschitz_bound)
                bound_soundness = br.soundness
                bound_method = br.method
                bound_confidence = br.confidence
            else:
                L = float(F_i.lipschitz(ctx))
                bound_soundness = getattr(F_i, "bound_soundness", "heuristic")
                bound_method = "static"
                bound_confidence = None

            slope_ub = (1.0 - alpha) + alpha * L
            proof_level, status = determine_certification(
                slope_ub, bound_soundness, ctx.contraction_tolerance
            )

            if status in ["certified", "accepted_heuristic"]:
                cert = CSPCertificate(
                    op_id=op.op_id,
                    step_index=ctx.step_index,
                    substep_index=ctx.substep_index,
                    timestamp=time.time(),
                    slope_ub=slope_ub,
                    gap_lb=1.0 - slope_ub,
                    alpha_used=alpha,
                    projection_residual=residual,
                    proof_level=proof_level,
                    status=status,
                    mode=ctx.mode,
                    profile=ctx.profile,
                    require_certificates=ctx.require_certificates,
                    norm_id=X.norm_id,
                    safety_set_id=X.safety_set_id,
                    projector_id=projector.projector_id,
                    fallback_index=fallback_idx,
                    fallback_op_id=F_i.op_id,
                    backtrack_steps=backtrack_count,
                    stagnation_count=ctx.alpha_controller.stagnant_count,
                    bound_soundness=bound_soundness,
                    bound_method=bound_method,
                    bound_confidence=bound_confidence,
                    repairs_applied=repairs,
                    assumptions=ctx.assumptions or [],
                    context_hash=compute_context_hash(X, ctx, op.op_id,
projector.projector_id),
```

```
                )

                # Controller update
                ctx.alpha_controller.step_accepted(alpha)
                if status == "accepted_heuristic":
                    ctx.heuristic_steps = (ctx.heuristic_steps or 0) + 1

                # Persist fallback index for future calls
                setattr(op, "_current_fallback_index", fallback_idx)
                return Y, cert

            # Rejection -> backtrack
            ctx.alpha_controller.step_rejected(alpha)
            alpha /= 2.0
            backtrack_count += 1

        ctx.alpha_controller.fallback_exhausted(fallback_idx)

    raise RuntimeError(f"CSPContractionError: {op.op_id} exhausted fallbacks")
```

---

## 10. Global contraction semantics

### 10.1 Pipeline proof summary

```
from dataclasses import dataclass
from typing import List, Literal, Optional

@dataclass
class PipelineProof:
    level: Literal["formal", "heuristic", "none"]
    global_slope_ub: Optional[float]
    weakest_link: Optional[int]

def summarize_pipeline(certificates: List[CSPCertificate]) -> PipelineProof:
    if not certificates:
        return PipelineProof(level="none", global_slope_ub=None,
weakest_link=None)

    any_freeze = any(c.status == "freeze" for c in certificates)
    all_formal = all(c.is_formally_certified for c in certificates)
    all_at_least_heuristic = all(c.status in ["certified", "accepted_heuristic"]
for c in certificates)

    if all_formal:
```

```
        log_slope = 0.0
        weakest = None
        weakest_gap = float("inf")
        for i, c in enumerate(certificates):
            log_slope += math.log(c.slope_ub)
            if c.gap_lb < weakest_gap:
                weakest_gap = c.gap_lb
                weakest = i
        return PipelineProof(level="formal",
global_slope_ub=math.exp(log_slope), weakest_link=weakest)

    if all_at_least_heuristic:
        return PipelineProof(level="heuristic", global_slope_ub=None,
weakest_link=None)

    if any_freeze:
        return PipelineProof(level="none", global_slope_ub=None,
weakest_link=None)

    return PipelineProof(level="none", global_slope_ub=None, weakest_link=None)
```

---

## 11. Deterministic replay: context hashing

The context hash MUST be deterministic for identical inputs and configuration. Timestamps are excluded from replay equivalence.

```
import hashlib
import json
import numpy as np
from typing import Optional

def compute_context_hash(
    X: TypedTensor,
    ctx: Context,
    op_id: str,
    projector_id: str,
    seed: Optional[int] = None,
) -> str:

    def tensor_to_bytes(arr: np.ndarray) -> bytes:
        arr = np.asarray(arr)
        scale = 1e9
        if np.iscomplexobj(arr):
            r = (np.real(arr) * scale).astype(np.int64)
```

```
            im = (np.imag(arr) * scale).astype(np.int64)
            return r.tobytes() + im.tobytes()
        q = (arr * scale).astype(np.int64)
        return q.tobytes()

    parts = []
    if hasattr(X.data, "shape"):
        parts.append(tensor_to_bytes(X.data))
    else:
        parts.append(str(X.data).encode("utf-8"))

    parts.append(json.dumps({
        "sigma": dict(X.sigma),
        "algebra": X.algebra,
        "norm_id": X.norm_id,
        "safety_set_id": X.safety_set_id,
    }, sort_keys=True).encode("utf-8"))

    parts.append(json.dumps({
        "op_id": op_id,
        "projector_id": projector_id,
        "mode": ctx.mode,
        "profile": ctx.profile,
        "alpha_min": ctx.alpha_controller.alpha_min,
        "k_stagnant": ctx.alpha_controller.k_stagnant,
        "contraction_tolerance": ctx.contraction_tolerance,
        "seed": int(seed if seed is not None else getattr(ctx, "seed", 0)),
        "allow_tightening": bool(ctx.allow_tightening),
        "require_certificates": bool(ctx.require_certificates),
    }, sort_keys=True).encode("utf-8"))

    parts.append(json.dumps({
        "alpha": float(ctx.alpha_controller.alpha),
        "stagnant_count": int(ctx.alpha_controller.stagnant_count),
        "rejection_count": int(ctx.alpha_controller.rejection_count),
    }, sort_keys=True).encode("utf-8"))

    return hashlib.sha256(b"".join(parts)).hexdigest()[:32]
```

---

# 12. Reference operators (starter set)

### 12.1 Affine operator (formal bound)

```
import numpy as np

class AffineOperator(Operator):
    bound_soundness = "formal"

    def __init__(self, op_id: str, A: np.ndarray, b: np.ndarray):
        self.op_id = op_id
        self.A = A
        self.b = b

    def __call__(self, X: TypedTensor, ctx: Optional[Context] = None) ->
TypedTensor:
        y = self.A @ np.asarray(X.data) + self.b
        return TypedTensor(y, X.sigma, X.algebra, X.norm_id, X.safety_set_id)

    def lipschitz(self, ctx: Context) -> float:
        # Spectral norm in l2
        return float(np.linalg.norm(self.A, 2))

    def lipschitz_max(self) -> float:
        return float(np.linalg.norm(self.A, 2))

    def fallback_chain(self) -> List[Operator]:
        scaled = AffineOperator(self.op_id + ":scaled0.5", 0.5 * self.A, 0.5 *
self.b)
        return [self, scaled, FreezeOperator()]
```

### 12.2 Depolarizing channel (formal in trace norm)

- Bind `norm_id="trace"` for certification.
- Contractivity factor for trace distance is $(1 - p)$ under standard conventions.

### 12.3 Product t-norm and MV-algebra modules

When implementing fuzzy operators, explicitly define the input packing and the product-space norm; do not assume `L=1` without a norm statement.

---

# 13. Validation plan (minimal, high-signal)

1. **Must-pass correctness tests**

2. type/FTS mismatch → fail-closed exception

3. projector returns `(Y, repairs, residual)` and residual is consistent with norm
4. formal vs heuristic labeling: no formal proof emitted from heuristic bounds
5. freeze semantics: returns unchanged `X`, `status="freeze"`, `proof_level="none"`

6. **Replay tests**

7. identical initial state/config → identical `context_hash` stream (timestamps ignored)

8. **Stability tests**

9. backtracking halts within bounded steps

10. stagnation triggers fallback escalation and resolves or freezes

11. **Pipeline proof tests**

12. all-formal pipeline → `PipelineProof.level="formal"` and global slope < 1

13. mixed heuristic pipeline → `PipelineProof.level="heuristic"`

---

# 14. External references (selected)

### Fixed point theory and non-expansive projections


- S. Banach, *Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales*, **Fundamenta Mathematicae** 3(1):133–181 (1922). DOI: 10.4064/fm-3-1-133-181.
- H. H. Bauschke, P. L. Combettes, *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*, Springer (2011). DOI: 10.1007/978-1-4419-9467-7.
- W. A. Kirk, B. Sims (eds.), *Handbook of Metric Fixed Point Theory*, Springer (2001). DOI: 10.1007/978-94-017-1748-9.


### Operator means and monotonicity (optional UL fusion algebra)


- F. Kubo, T. Ando, *Means of positive linear operators*, **Mathematische Annalen** 246:205–224 (1980). DOI: 10.1007/BF01371042.


### Quantum channels, Choi representations, and diamond / CB norms


- M.-D. Choi, *Completely positive linear maps on complex matrices*, **Linear Algebra and its Applications** 10(3):285–290 (1975). DOI: 10.1016/0024-3795(75)90075-0.
- J. Watrous, *Semidefinite programs for completely bounded norms*, **Theory of Computing** 5(11) (2009). DOI: 10.4086/toc.2009.v005a011.

• J. Watrous, *The Theory of Quantum Information*, Cambridge University Press (2018). (See chapters on completely bounded trace norm / diamond norm.)

### Many-valued logics and MV-algebras (for fuzzy modules)

• R. L. O. Cignoli, I. M. L. D'Ottaviano, D. Mundici, *Algebraic Foundations of Many-Valued Reasoning*, Kluwer/Springer (2000). DOI: 10.1007/978-94-015-9480-6.
• S. A. Kripke, *Semantical Analysis of Intuitionistic Logic I*, in **Formal Systems and Recursive Functions** (North-Holland, 1965). (Kripke semantics for Heyting/intuitionistic logic.)

### Effect algebras and unsharp quantum logic (for quantum-effect modules)

• D. J. Foulis, M. K. Bennett, *Effect algebras and unsharp quantum logics*, **Foundations of Physics** 24(10): 1331–1352 (1994). DOI: 10.1007/BF02283036.

---